

MOVIE RATING PREDICTION PROJECT

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
```

```
In [4]: df = pd.read_csv("IMDb Movies India.csv", encoding='latin1')

print(df.head())
```

	Name	Year	Duration	Genre	\
0		NaN	NaN	Drama	
1	#Gadhvi (He thought he was Gandhi)	(2019)	109 min	Drama	
2	#Homecoming	(2021)	90 min	Drama, Musical	
3	#Yaaram	(2019)	110 min	Comedy, Romance	
4	...And Once Again	(2010)	105 min	Drama	

	Rating	Votes	Director	Actor 1	Actor 2	\
0	NaN	NaN	J.S. Randhawa	Manmauji	Birbal	
1	7.0	8	Gaurav Bakshi	Rasika Dugal	Vivek Ghamande	
2	NaN	NaN	Soumyajit Majumdar	Sayani Gupta	Plabita Borthakur	
3	4.4	35	Ovais Khan	Prateik	Ishita Raj	
4	NaN	NaN	Amol Palekar	Rajat Kapoor	Rituparna Sengupta	

	Actor 3
0	Rajendra Bhatia
1	Arvind Jangid
2	Roy Angana
3	Siddhant Kapoor
4	Antara Mali

```
In [5]: print(df.info())

print(df.isnull().sum())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 15509 entries, 0 to 15508
Data columns (total 10 columns):
 #   Column      Non-Null Count  Dtype
---  -
 0   Name        15509 non-null  object
 1   Year        14981 non-null  object
 2   Duration    7240 non-null   object
 3   Genre       13632 non-null  object
 4   Rating      7919 non-null   float64
 5   Votes       7920 non-null   object
 6   Director    14984 non-null  object
 7   Actor 1     13892 non-null  object
 8   Actor 2     13125 non-null  object
 9   Actor 3     12365 non-null  object
dtypes: float64(1), object(9)
memory usage: 1.2+ MB
None
Name          0
Year          528
Duration      8269
Genre         1877
Rating        7590
Votes         7589
Director       525
Actor 1       1617
Actor 2       2384
Actor 3       3144
dtype: int64
```

```
In [6]: df.dropna(inplace=True)

print(df.isnull().sum())
```

```
Name          0
Year          0
Duration       0
Genre          0
Rating         0
Votes          0
Director       0
Actor 1        0
Actor 2        0
Actor 3        0
dtype: int64
```

```
In [7]: le = LabelEncoder()

df['Genre'] = le.fit_transform(df['Genre'])
df['Director'] = le.fit_transform(df['Director'])
df['Actor 1'] = le.fit_transform(df['Actor 1'])
df['Actor 2'] = le.fit_transform(df['Actor 2'])
df['Actor 3'] = le.fit_transform(df['Actor 3'])
```

```
In [8]: x = df[['Genre', 'Director', 'Actor 1', 'Actor 2', 'Actor 3']]
y = df['Rating']
```

```
In [9]: x_train, x_test, y_train, y_test = train_test_split(
        x, y, test_size=0.2, random_state=42
    )
```

```
In [10]: model = LinearRegression()

         model.fit(x_train, y_train)
```

```
Out[10]: ▼ LinearRegression ⓘ ?
         ► Parameters
```

```
In [11]: prediction = model.predict(x_test)

         print(prediction[:5])

[5.65833159  6.01425693  5.91972685  5.73443558  5.65461656]
```

```
In [12]: mse = mean_squared_error(y_test, prediction)

         r2 = r2_score(y_test, prediction)

         print("Mean Squared Error:", mse)

         print("R2 Score:", r2)
```

Mean Squared Error: 1.824204245176786
R2 Score: 0.014872145511032775

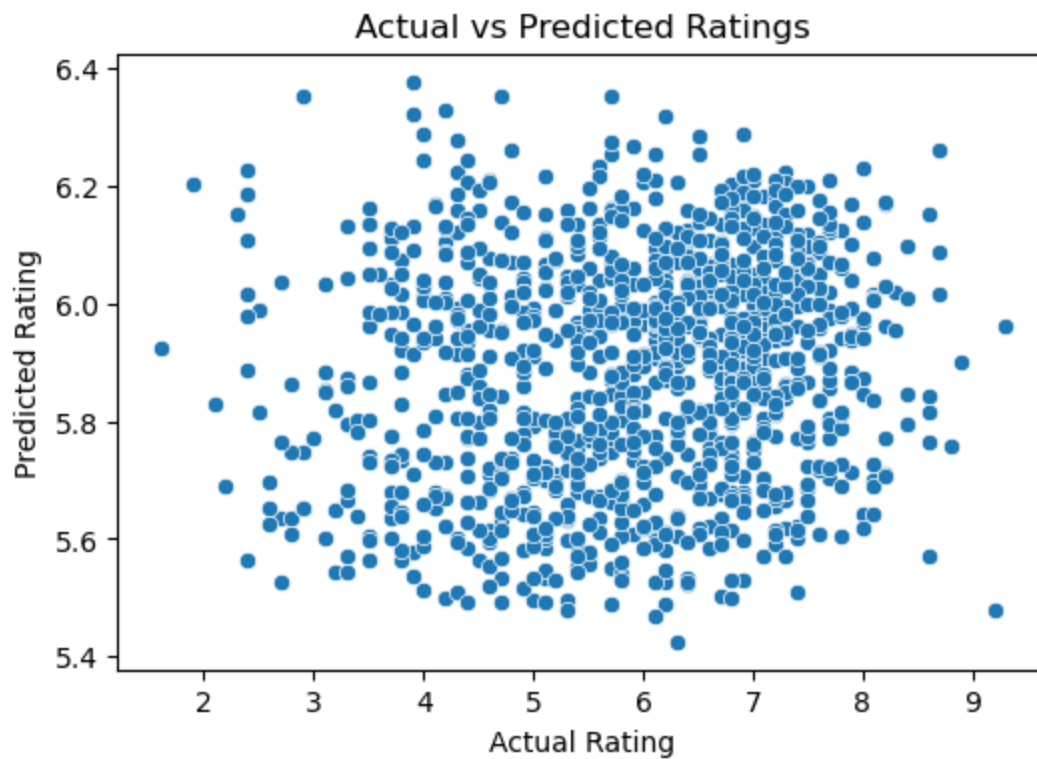
```
In [13]: plt.figure(figsize=(6,4))

         sns.scatterplot(x=y_test, y=prediction)

         plt.xlabel("Actual Rating")
         plt.ylabel("Predicted Rating")

         plt.title("Actual vs Predicted Ratings")

         plt.show()
```



```
In [14]: print("Movie Rating Prediction Project Completed Successfully")
```

Movie Rating Prediction Project Completed Successfully

```
In [ ]:
```